

Experiment No.	4
Experiment Title	Logistic Regression and Kernel SVM for Spam Email Classification ¹
Student Name	S.R.Aadhithya
Register Number	3122247001001
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02-08-2026

1 Objective

This experiment compares Logistic Regression against Support Vector Machines (with linear, polynomial, RBF, and sigmoid kernels) on the Spam-base spam-classification task, this time with a shared log-transform-and-scale preprocessing step instead of a bespoke transform per model. Both models are tuned with `GridSearchCV`, and the four SVM kernels are additionally compared kernel-by-kernel on accuracy, F1-score, and training time to see whether the extra flexibility of a non-linear kernel is actually worth its computational cost here.

2 Problem Statement

The task is binary classification: given 57 numeric features describing word frequencies, character frequencies, and capital-letter run lengths in an email, predict whether it's spam (1) or not (0). The input is the same fully numeric Spambase feature matrix; the output is a binary label, evaluated with accuracy, precision, recall, and F1-score (with F1 specifically used as the scoring metric during hyperparameter search).

3 Dataset Description

Dataset Name	Spambase (<code>spambase_csv.csv</code>)
Dataset Source	UCI Machine Learning Repository
Number of Samples	4,601 emails
Number of Features	57 (48 word-frequency, 6 character-frequency, 3 capital-run-length features)
Number of Classes	2 - spam (1) vs. not spam (0); roughly a 39/61 split
Missing Values	0 - confirmed directly via <code>df.isnull().sum()</code>
Train-Test Split	80 / 20, stratified by class

4 Exploratory Data Analysis

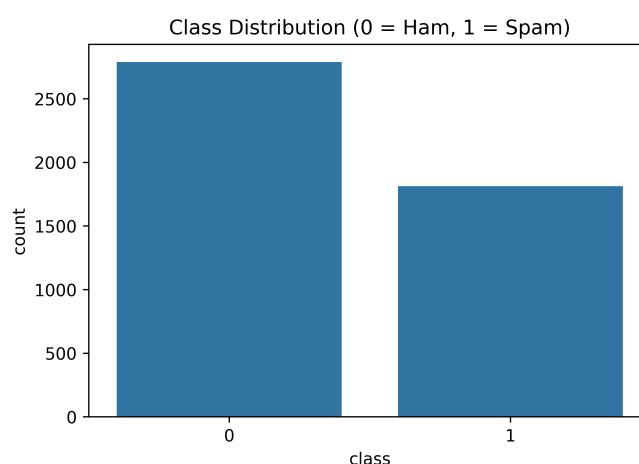


Figure 1: Class distribution - 0 = Ham, 1 = Spam

Inference: Consistent with Experiment 2's EDA on the same dataset, non-spam ("ham") emails outnumber spam by roughly 3:2 rather than anything close to a 50/50 split. It's mild enough that raw accuracy remains informative here, but it's still the reason F1-score was used as the scoring metric for both grid searches rather than accuracy alone.

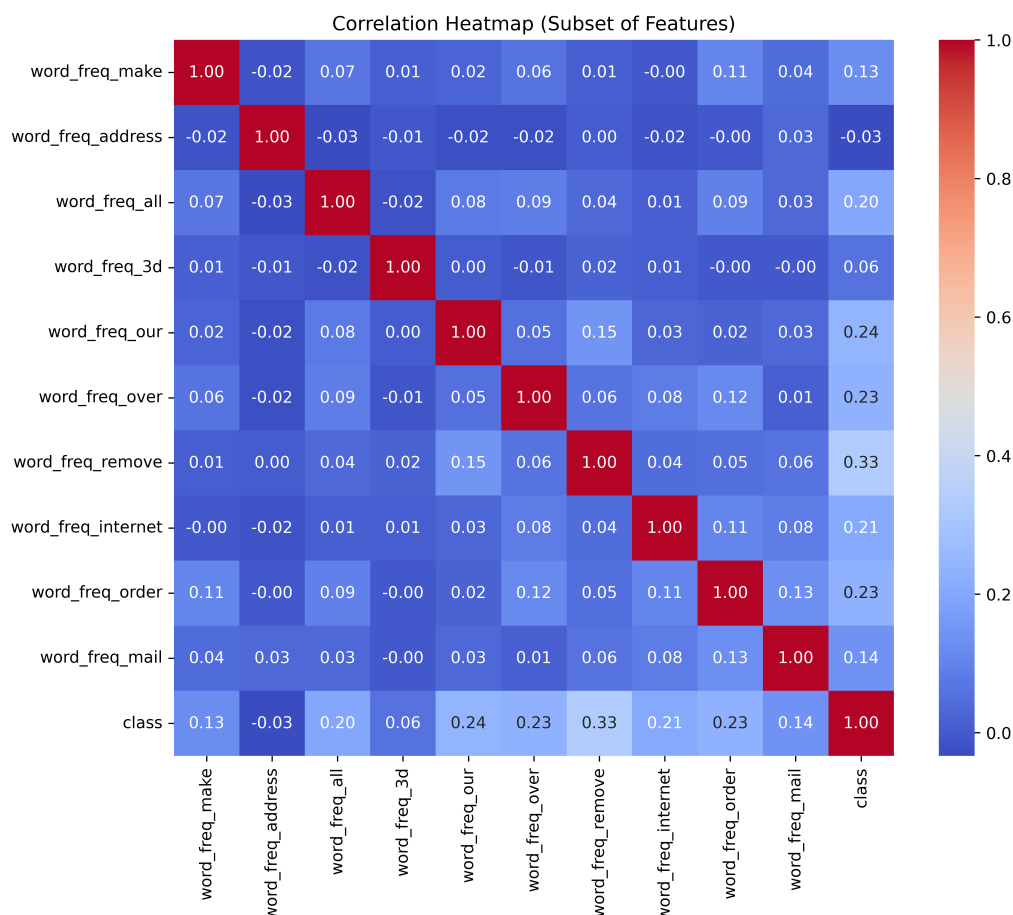


Figure 2: Annotated correlation heatmap - first 10 features and the class label

Inference: Restricting the heatmap to 10 features and annotating each cell trades broad coverage for readability - you can actually read individual correlation values here, unlike a full 57-feature heatmap. Words directly associated with promotional email content (`word_freq_our`, `word_freq_remove`, `word_freq_internet` among this subset) tend to show a noticeably positive relationship with the spam label, while a more generic word like `word_freq_address` tends to sit close to zero - in line with which words you'd intuitively expect a spam filter to key on.

5 Data Preprocessing

No missing values were found. Rather than treating Logistic Regression and SVM differently, a single shared preprocessing pipeline was used for both: a $\log(1 + x)$ transform (via `FunctionTransformer(np.log1p)`, which safely handles the many zero values in the word-frequency columns) followed by `StandardScaler`. This directly targets the same zero-inflation

and long right tail that Experiment 2's EDA identified in these features, compressing the extreme values before scaling rather than leaving them to dominate the distance/decision-boundary calculations that both Logistic Regression and SVM rely on. The 80/20 train-test split was stratified by class this time, ensuring both splits keep the same roughly 39/61 spam/ham ratio as the full dataset.

6 Machine Learning Algorithm

6.1 Logistic Regression

Logistic Regression models the log-odds of the positive class as a linear function of the features, passed through a sigmoid:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^\top x + b)}}$$

Coefficients are fit by maximizing the likelihood of the observed labels (equivalently, minimizing log-loss), optionally with an L1 or L2 penalty on w to control overfitting.

Advantages: fast to train, coefficients are directly interpretable as each feature's contribution to the log-odds, and an L1 penalty can zero out uninformative features automatically.

Limitations: it can only learn a linear decision boundary in the (transformed) feature space - if spam and ham aren't linearly separable even after the log-transform, Logistic Regression has no way to bend the boundary to compensate.

6.2 Support Vector Machine (Kernel Methods)

An SVM finds the hyperplane that maximizes the margin between classes. For non-linearly-separable data, a kernel function implicitly maps the input into a higher-dimensional space where a linear separator may exist, without ever computing that mapping explicitly:

$$K(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$$

The RBF kernel used as the default here is $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$, which effectively measures similarity as a smooth function of distance.

Advantages: the kernel trick lets SVM capture non-linear decision boundaries (RBF, polynomial) without explicitly engineering non-linear features;

the margin-maximizing objective tends to generalize well even with a fairly modest number of support vectors.

Limitations: more hyperparameters to tune than Logistic Regression (kernel choice, C , γ , and for polynomial kernels, degree), slower to train as the dataset grows, and - like Logistic Regression - sensitive to feature scaling, which is exactly why the same `log1p + StandardScaler` pipeline was applied before fitting either model.

7 Experimental Setup

Python Version	
Scikit-Learn Version	
IDE	
Operating System	
Hardware	
Environment Activation	<code>source /opt/anaconda3/bin/ activate anaconda-navigation</code>

8 Hyperparameter Selection

Both models were tuned with `GridSearchCV` (5-fold cross-validation, scored on F1).

Logistic Regression grid:

Parameter	Grid Searched
C	$\{0.01, 0.1, 1, 10, 100\}$
penalty	$\{l1, l2\}$
solver	$\{\text{liblinear}, \text{saga}\}$

Best found: $C = 100$, `penalty='l1'`, `solver='liblinear'` (Best CV F1: 0.9244).

SVM grid (searched as three separate parameter sets, one per kernel family):

Kernel	Grid Searched
linear	$C \in \{0.1, 1, 10, 100\}$
poly	$C \in \{0.1, 1, 10, 100\}$, $\gamma \in \{\text{scale}, \text{auto}\}$, $\text{degree} \in \{2, 3, 4\}$
rbf / sigmoid	$C \in \{0.1, 1, 10, 100\}$, $\gamma \in \{\text{scale}, \text{auto}\}$

Best found overall: `kernel='rbf'`, $C = 10$, $\gamma = 'auto'$ (Best CV F1: 0.9357).

Note: re-running the Logistic Regression grid on a newer scikit-learn version may raise a `FutureWarning` - the `penalty` parameter is being deprecated in favour of `l1_ratio`, so the grid above may need updating to $l1_ratio \in \{0, 1\}$ instead of `penalty` in a future revision of this notebook.

9 Results

Table 1 - Baseline (Untuned) Comparison:

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
Logistic Regression (default)	0.94	0.93	0.93	0.93
SVM, RBF ($C = 1.0$, default γ)	0.94	0.94	0.94	0.94

Table 2 - Tuned Logistic Regression, Final Performance:

Metric	Value
Accuracy	0.9338
Precision	0.9194
Recall	0.9118
F1-score	0.9156
Training Time (s)	0.1083

Table 3 - SVM, Kernel-wise Performance (best hyperparameters per kernel):

Kernel	Accuracy	F1-score	Training Time (s)
Linear	0.9305	0.9111	0.3546
Poly	0.9349	0.9150	0.1602
RBF	0.9457	0.9306	0.1756
Sigmoid	0.9381	0.9201	0.2466

Table 4 - 5-Fold Cross-Validation, Best Logistic Regression vs. Best SVM:

Fold	Logistic Regression	SVM
1	0.9567	0.9432
2	0.9220	0.9393
3	0.9197	0.9422
4	0.9072	0.9171
5	0.9164	0.9366
Average	0.9244	0.9357

The RBF-kernel SVM comes out ahead of the tuned Logistic Regression on every metric that both were measured on - accuracy (0.9457 vs. 0.9338), F1-score (0.9306 vs. 0.9156), and average cross-validation F1 (0.9357 vs. 0.9244). One detail worth a second look: the *tuned* Logistic Regression's test accuracy (0.9338, rounding to 0.93 in the classification report) is actually slightly *lower* than the untuned default model's test accuracy (0.94) from Table 1, despite the tuned version having the better cross-validated F1-score during grid search. That's not a sign the tuning failed - it's a reminder that a model chosen for the best average performance across 5 CV folds isn't guaranteed to also win on one specific held-out test split, especially once the performance differences involved are this small.

10 Result Visualization

This notebook doesn't generate confusion matrices, ROC curves, or other post-training visualizations beyond the two EDA plots already shown above - all model comparison here is done through the classification reports and summary tables in the Results section. If this experiment were extended, confusion matrices for the tuned Logistic Regression and each SVM kernel would be a natural next addition, to see whether the accuracy differences in Table 3 are concentrated in false positives, false negatives, or both.

11 Discussion

A key structural aspect of this implementation is the preprocessing: instead of a different transform for every model (as Naive Bayes required), a single `log1p + StandardScaler` pipeline served both Logistic Regression and SVM here, since both are scale-sensitive and neither needs the strictly non-negative or binary-valued inputs that Multinomial/Bernoulli Naive

Bayes did. That simplicity is a genuine advantage of sticking to models with similar input assumptions.

On the model comparison itself, RBF-kernel SVM wins clearly and consistently - best CV F1, best test accuracy, best test F1 among all configurations tried. The margin isn't huge (roughly 1–1.5 points of accuracy and F1 over the tuned Logistic Regression), which makes sense given the `log1p` transform already does a lot of the work in making these features closer to linearly separable - a purely linear model was never going to be far behind here.

The kernel-wise timing comparison in Table 3 has an interesting wrinkle: the linear kernel - the simplest of the four mathematically - took the longest to train (0.355s), noticeably more than poly or RBF (0.16–0.18s). That's a reminder that theoretical simplicity doesn't always translate to faster fitting in practice; the specific hyperparameters chosen per kernel and how well `libsvm`'s solver behaves on that particular combination matter more than the kernel's conceptual complexity.

The one result worth flagging as a genuine caveat rather than just an interesting aside is the tuned Logistic Regression underperforming its own untuned baseline on the single test split (Table 1 vs. Table 2). This is most likely ordinary variance between the grid search's internal 5-fold CV and one fixed 80/20 split, rather than a real regression - but it's exactly the kind of result that would be worth double-checking with a repeated/nested cross-validation setup before trusting the "tuned" label at face value.

12 Conclusion

Between the two families of model tried here, RBF-kernel SVM ($C = 10$, $\gamma = \text{'auto'}$) gave the best overall spam-classification performance - 94.57% test accuracy and an F1-score of 0.9306, against 93.38% accuracy and an F1-score of 0.9156 for the best tuned Logistic Regression. The gap is real but modest, and comes with a training-time cost of roughly 60% more than the tuned Logistic Regression (0.176s vs. 0.108s). Whether that trade-off is worth it in practice depends on the deployment setting: if training happens rarely and inference speed matters more, the interpretability and speed of Logistic Regression remain attractive despite the slightly lower F1-score. A natural next step would be re-running the Logistic Regression grid with `l1_ratio` instead of the deprecated `penalty` parameter, and adding confusion matrices per model to see exactly where the remaining errors are concentrated.